From Naive Loops to a Packed Microkernel: An Empirical, Reproducible Study of Single-Threaded Matrix Multiplication on a Hybrid x86 Laptop CPU

Pratham Srivastava
Department of Information Technology
Noida Institute of Engineering and Technology (NIET)
Greater Noida, Uttar Pradesh, India

contact.pratham21@gmail.com

***Abstract*—General matrix multiplication (GEMM) dominates the runtime of dense linear algebra and neural-network workloads, yet the same arithmetic can run hundreds of times faster or slower depending on how it uses caches, SIMD units and registers. This paper reports a step-by-step, pre-registered experimental study of single-threaded GEMM written in C++20 and measured on an Intel Core i5-13450HX laptop processor. Five kernel versions are built in sequence: a naive i-j-k loop, all six loop orders, cache blocking, explicit AVX2/FMA intrinsics, and a Goto/BLIS-style packed kernel with a 6×16 register-blocked microkernel. Every kernel is verified before timing, every prediction is written down before the run, and all raw measurements are published. The naive kernel collapses by about 5× when the matrix row stride is exactly 4 KiB; loop order alone spans up to 85×; cache blocking gains up to 4.0× over the best unblocked order; hand-written AVX2 intrinsics are 1.5–2.0× faster than SSE2-only compiler output but do not beat the compiler's own AVX2 auto-vectorization. The packed microkernel reaches 125 GFLOP/s in single precision (about 43% of the estimated single-core peak), 2.5–3.5× above the best compiler-generated kernel measured in the same run, and cuts the power-of-two-size penalty from 23–32% to under 8%. Two of five predictions held fully and three held partially; the misses, and a 1.4× day-to-day drift in absolute throughput, are reported as findings.**

***Index Terms*—matrix multiplication, GEMM, cache blocking, SIMD, AVX2, FMA, register blocking, performance engineering, reproducibility.**

I. Introduction

Multiplying two dense matrices, C = AB, is the central kernel of the Level-3 BLAS [1] and of most deep-learning training and inference. Its arithmetic is fixed at $2n^3$ floating-point operations for n×n matrices, but its speed is not: a modern core can only reach its arithmetic peak if operands arrive from the memory hierarchy fast enough, and naive code rarely lets that happen. High-performance libraries close this gap with a well-known set of techniques—loop ordering, cache blocking, vectorization, register blocking and packing [2], [3]—but the size of each technique's contribution on a given machine is usually taken on trust.

This study measures those contributions directly. Starting from a textbook triple loop, the kernel is optimized one idea at a time on a single, recent consumer laptop CPU, and each step is treated as an experiment: a written prediction, a controlled run, and a recorded outcome, including the cases where the prediction was wrong. The emphasis is on methodology as much as on speed: every timing is preceded by a correctness check, every run records the exact code version and build flags, and the raw data are committed to a public repository [14].

The contributions are: (1) a quantified optimization ladder (V0–V4) for single-threaded float and double GEMM on an Intel Raptor Lake-HX processor; (2) a cache-set explanation of a 5× performance collapse at power-of-two sizes and evidence that packing removes it; (3) a negative result—hand-written AVX2 intrinsics do not beat the compiler's auto-vectorization unless the data layout also changes; and (4) a lightweight, pre-registered benchmarking protocol suitable for student research. Multithreading and a comparison with an optimized BLAS library are outside the scope of this paper and are planned as future work.

II. Background and Related Work

A. Memory Hierarchy and Conflict Misses

Caches hold recently used 64-byte lines in a limited number of sets; a line's set is fixed by its address. When a loop walks down a matrix column, consecutive accesses are one row stride apart. If the stride is a large power of two, the accesses map to very few sets and evict each other even though the cache is mostly empty. Lam, Rothberg and Wolf [5] analyzed this self-interference for blocked algorithms. For rows that start on line boundaries and a stride of s cache lines, a column touches

$$\text{sets spanned} = 64 / \gcd(s, 64) \quad (1)$$

of the 64 L1 sets of the processor studied here, so a 4096-byte stride ($s = 64$) confines an entire column to a single 12-way set.

B. Blocking, Vectorization and Register Blocking

Cache blocking (tiling) reorders the computation so that small tiles of A, B and C are reused while they are still cached [5], [7]. SIMD extensions such as AVX2 process 8 single- or 4 double-precision values per instruction, and fused multiply-add (FMA) performs a multiply and an add in one instruction. Goto and van de Geijn [2] showed that the remaining gap to peak is closed by (i) keeping a small MR×NR tile of C in vector registers across a long inner loop (the microkernel) and (ii) copying (packing) blocks of A and B into contiguous buffers ordered exactly as the microkernel reads them. BLIS [3] structures this as five nested loops, and Low et al. [4] showed that the block sizes can be derived analytically from cache parameters, which is the approach followed here. The roofline

model [6] frames why reducing memory traffic per FMA matters more than raw instruction count.

C. Benchmarking Methodology

Hoefler and Belli [10] list common reporting failures in performance studies, including unreported variability, missing configuration details and unverified results. The protocol below addresses these with medians and interquartile ranges, full build metadata per run, and Freivalds' randomized verification [8], which checks a product in $O(n^2)$ time using a floating-point error bound of the standard form [9].

III. Experimental Setup and Methodology

A. Platform

All experiments ran on one laptop (Table I). The processor is a hybrid design with six performance (P) cores and four efficiency (E) cores. Under WSL2 the operating system exposes a uniform view of the caches, so the P-core values from lscpu were used to size blocks; core placement could not be controlled or observed.

TABLE I
Experimental Platform and Measurement Settings

Item	Value
CPU	Intel Core i5-13450HX: 6 P-cores (2-way SMT) + 4 E-cores, 16 logical CPUs
P-core caches	L1d 48 KiB, 12-way, 64 sets; L2 1.25 MiB; L3 20 MiB shared
Max. turbo	4.6 GHz (P-core) [13]; actual frequency not measured
Memory / OS	24 GB; Windows 11 + WSL2 (Linux 6.6.87.2-microsoft-standard)
Compiler	GCC 15.2.0, C++20, Release (-O3 -DNDEBUG)
Builds	Portable (x86-64 baseline, SSE2); native (-march=native, AVX2/FMA)
Conditions	AC power, Windows "Best performance" mode (script-enforced)
Timing	steady_clock (1 ns); samples ≥ 50 ms; 5 reps (7 for EXP01); median
Correctness	14 unit tests per build; Freivalds check before every timing

B. Kernel Ladder

Table II lists the kernels. All share one interface (row-major A, B, C; C overwritten) and are registered in a single table that the tests and the benchmark both iterate over, so no kernel can be timed without first being tested. V0–V2 are plain C++ that the compiler is free to vectorize. V3 and V4 use AVX2/FMA intrinsics enabled per function (GCC target attributes), and are registered only if a run-time CPUID/XGETBV check confirms that the CPU and operating system support AVX2 and FMA; the rest of the library therefore stays portable. Disassembly confirmed 17 FMA instructions in the V3 object file and none in the portable V2 file.

TABLE II
Kernels Studied

Ver.	Name	Description
V0	reference	Textbook i-j-k triple loop
V1	loop_xyz	The other five loop orders (ikj, jik, jki, kij, kji)
V2	blocked_bBS	BS×BS tiles (ii-kk-jj), i-k-j inside a tile; BS ∈ {16, 32, 48, 64, 96, 128, 192, 256}
V3	avx2_b64	V2 with BS = 64 and an AVX2/FMA intrinsic inner loop: broadcast a(i,k), load/FMA/store a vector of C
V4	packed	Goto/BLIS five-loop kernel: packed, zero-padded A and B panels; 6×16 (float) / 6×8 (double) microkernel in 12 YMM accumulators; KC = 256, MC = 288, NC = 4096

The V4 block sizes follow the analytical rules of [4]. One A micro-panel plus one B micro-panel must fit in about half of L1: $(6+16)\cdot256\cdot4$ B ≈ 22.5 KiB for float and $(6+8)\cdot256\cdot8$ B ≈ 28 KiB for double. The packed A block (MC×KC) targets L2: $288\cdot256\cdot8$ B = 576 KiB for double. The packed B block (KC×NC) targets L3: $256\cdot4096\cdot8$ B = 8 MiB. In the microkernel each k-step issues two vector loads of B, six broadcasts of A and twelve independent FMAs, enough to cover the FMA latency on two FMA ports; C is read and written only once per KC-deep pass.

C. Measurement Protocol

For each square size n (outer loop) and kernel (inner loop, so kernels compared at one size run back to back), the harness allocates and fills the inputs outside the timed region, makes one untimed warm-up call, and verifies its output with Freivalds' test; a kernel that fails is never timed. The number of calls per sample is calibrated so that each sample lasts at least 50 ms. Throughput is reported as

$$\text{GFLOP/s} = 2n^3 / t \tag{2}$$

where t is the time per call in nanoseconds. Each run writes every sample to a CSV file and a JSON sidecar with the git commit, CPU model and features, compiler, flags and build type. Experiment scripts rebuild both configurations from scratch, run the unit tests, refuse to start on battery power, and label runs made from uncommitted code. Across all 316 configurations reported here the median interquartile range is 1.6% of the median, and 89% of configurations are below 5%; the exceptions are discussed where they occur.

D. Research Protocol

Before each formal run, a one-paragraph prediction was written in a lab notebook and committed; the result was recorded afterwards against it. Exploratory ("scratch") runs are kept separate and never cited. One run was discarded because a stale binary from older code was executed; the fresh-rebuild rule above was introduced in response. The prediction for EXP05 was written after a single scratch sanity run, which is disclosed here.

E. Peak Estimate

A Raptor Lake P-core can issue two 256-bit FMAs per cycle. At the 4.6 GHz maximum turbo [13] this gives $2 \times 8 \times 2 \times 4.6 = 294.4$ GFLOP/s per core in single precision and 147.2 GFLOP/s in double precision. Because the sustained clock under WSL2 could not be measured, fractions of peak below are approximate upper-bound normalizations.

IV. Results

A. EXP01: Power-of-Two Stride Conflicts

Fig. 1 and Table III show the naive kernel near n = 512 (double) and n = 1024 (float). Both cases have a row stride of exactly 4096 bytes, and both collapse: double drops from 4.61 GFLOP/s at n = 511 to 0.92 at n = 512 (5.0×), and float from

4.17 at n = 1023 to 0.79 at n = 1024 (5.4×). Smaller dips at n = 496 and 528 (s = 62 and 66, 32 sets) relative to 504 and 520 (s = 63 and 65, all 64 sets) match the set-mapping model (1). A repeat in Windows power-saving mode was about 1.45× slower overall but showed the same pattern, so power mode affects absolute throughput but not the effect; this is why the standard conditions fix it. The prediction held. Hardware counters were not available under WSL2, so TLB effects cannot be excluded.

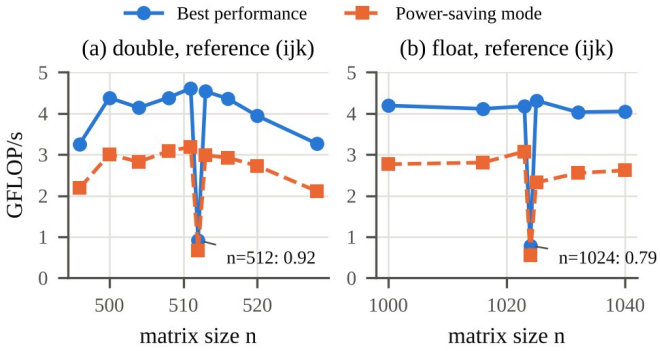


Fig. 1. EXP01. Naive (ijk) kernel throughput around the 4 KiB row-stride sizes. Solid: standard conditions (median of 7). Dashed: same experiment in Windows power-saving mode. Portable build.

TABLE III
EXP01: ROW STRIDE, CACHE SETS SPANNED AND THROUGHPUT (DOUBLE, REFERENCE)

n	Stride (B)	s (lines)	Sets spanned	GFLOP/s
496	3968	62	32	3.26
504	4032	63	64	4.14
511	4088	—	—	4.61
512	**4096**	**64**	**1**	**0.92**
513	4104	—	—	4.54
520	4160	65	64	3.94
528	4224	66	32	3.27

Model (1) applies only when the stride is a whole number of 64-byte lines ("—" otherwise). Runs 20260924-212250 (double) and 20260924-212257 (float).

B. EXP02: Loop Order

All six loop orders perform identical arithmetic, yet their throughput differs by up to 85× (Fig. 2, Table IV). As predicted, ikj and kij are fastest, because their innermost loop streams contiguous rows of B and C, and jki and kji are slowest, because they walk columns of A and C. With -march=native the compiler vectorized ikj with AVX2, raising it 1.5–2.7× (float) and 1.3–2.5× (double) over the portable build, while ijk and jik did not change: their inner loop is a sum reduction, which GCC does not reorder across SIMD lanes without permission to reassociate floating-point additions. The best-to-worst gap grows with n, from 29× at n = 128 to 85× at n = 1024 (native float). Power-of-two cliffs appeared only in the column-walking orders. One prediction failed: ikj and kij are not equal at large n (22.7 vs. 13.9 GFLOP/s, native float, n = 1024), because kij streams all of C once per value of k while ikj keeps one row of C in cache across the whole k loop.

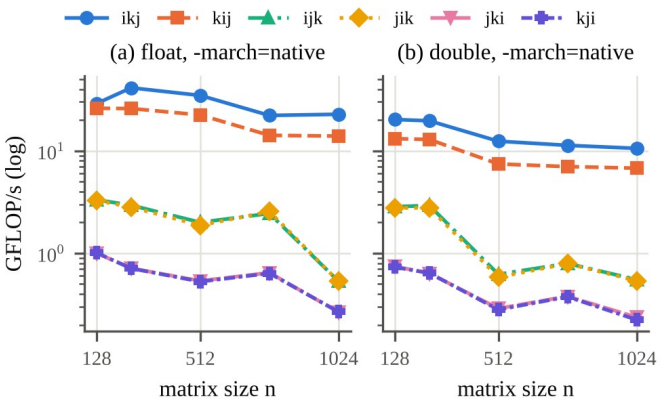


Fig. 2. EXP02. All six loop orders, native build, log scale (median of 5). ijk and jik overlap, as do jki and kji. Run 20261009-063213.

TABLE IV
EXP02: LOOP-ORDER THROUGHPUT, FLOAT (GFLOP/S)

Order	Portable 256	Native 256	Portable 1024	Native 1024
ikj	15.50	**41.18**	15.46	**22.71**
kij	18.43	25.83	12.19	13.91
ijk (V0)	2.81	2.94	0.51	0.53
jik	2.80	2.82	0.53	0.53
jki	0.71	0.71	0.24	0.27
kji	0.72	0.71	0.27	0.27

Runs 20261009-062519 (portable) and 20261009-063213 (native).

C. EXP03: Cache Blocking

Fig. 3 sweeps the tile size. Throughput has an interior optimum at BS = 64–96 for both precisions, as predicted: very small tiles (BS = 16) are slower than unblocked ikj because the inner loop is too short, and very large tiles no longer fit in cache. The benefit of blocking grows with n (Table V), reaching 4.0× for double at n = 2000, where unblocked ikj collapses to 4.5 GFLOP/s once B no longer fits in L2. Power-of-two sizes stay 27% (float) to 32% (double) slower than n = 2000 even with the best tile, and large tiles degrade most (float BS = 256: 30.9 at n = 2000 vs. 17.8 at n = 2048), consistent with self-interference between tile rows exactly 8 or 16 KiB apart [5]. The prediction held.

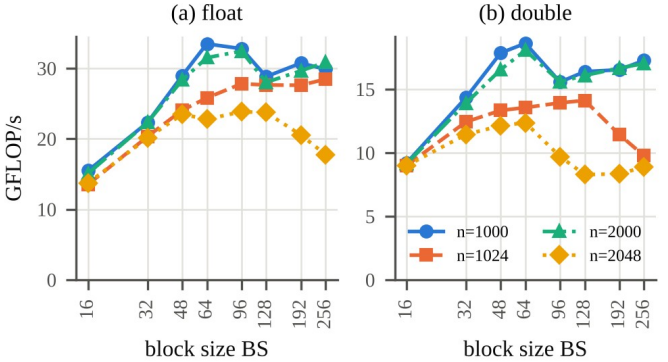


Fig. 3. EXP03. Throughput of the blocked kernel (V2) versus tile size, native build (median of 5). Run 20261009-063956.

TABLE V
EXP03: BEST TILE SIZE VERSUS UNBLOCKED IKJ (GFLOP/S)

Type, n	ikj	Best BS	Blocked	Speedup
float, 1000	23.02	64	33.47	1.45×
float, 1024	22.69	256	28.46	1.25×
float, 2000	14.49	96	32.44	2.24×
float, 2048	15.07	96	23.84	1.58×
double, 1000	10.87	64	18.60	1.71×

Type, n	ikj	Best BS	Blocked	Speedup
double, 1024	10.10	128	14.12	1.40×
double, 2000	4.49	64	18.06	**4.02×**
double, 2048	4.53	64	12.36	2.73×

D. EXP04: Explicit AVX2/FMA Intrinsics

V3 replaces only the innermost loop of the BS = 64 blocked kernel with AVX2/FMA intrinsics. Fig. 4 compares it with the compiler-generated blocked kernel in both builds. Against the portable build, which is limited to 128-bit SSE2 without FMA, the intrinsic kernel is 1.5–2.0× faster in nine of ten configurations, as predicted. Against the native build, however, the compiler's own AVX2 code is equal or faster in nine of ten configurations: the V3/V2 ratio is 0.81–0.94 for float and 0.86–1.03 for double. The prediction of a tie within 10% therefore held only partially; the largest gaps occur at n = 1000 and 2000, and near-ties at the power-of-two sizes, where both kernels are limited by the same conflict misses. A plausible but unverified explanation is that GCC unrolls its vectorized loop while the intrinsic loop issues one load–FMA–store per iteration. More fundamentally, both kernels load and store a vector of C for every FMA, so neither can approach peak; this motivates V4.

Because V3 compiles to the same code in both builds, its two measurements also bound run-to-run variation: they agree within 16%, except portable float n = 512 (16.6 vs. 24.1 GFLOP/s). That configuration was the first measured after the build and had an IQR of 74%, with individual samples between 13.3 and 27.2 GFLOP/s, which points to a frequency ramp-up rather than a property of the kernel. A side observation: at n = 512, native unblocked ikj (34.5 GFLOP/s) beats every blocked kernel, because the 1 MiB B matrix already fits in L2 and tiling only adds overhead.

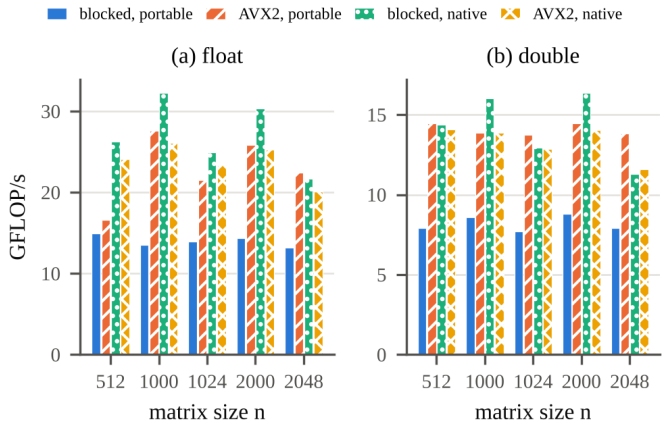


Fig. 4. EXP04. Compiler-generated blocked kernel (V2, BS = 64) versus the hand-written AVX2/FMA version (V3) in the portable and native builds (median of 5). Runs 20261009-090605 and 20261009-090853.

E. EXP05: Packed Register-Blocked Microkernel

V4 is the largest single step of the ladder (Fig. 5, Table VI). In single precision it reaches 125 GFLOP/s at n = 1000 and 127 GFLOP/s at n = 512, about 43% of the estimated peak, up from roughly 11% for the best V2/V3 kernels in EXP04. Measured in the same run, it is 2.5–3.5× faster than the compiler-vectorized blocked kernel for float and 1.9–2.9× for double, below the predicted 3–5×. The baseline itself, however, ran about 1.4× faster than in EXP04 (float n = 1000: 46.3 vs. 32.3 GFLOP/s), and so did the unchanged V3 kernel (37.2 vs. 26.1); since identical code sped up by the same factor, the drift reflects machine state (for example clock or thermal conditions) rather than the code. Against the EXP04 baseline the ratio would be about 3.9×. The practical lesson is that kernels should only be compared within a single run.

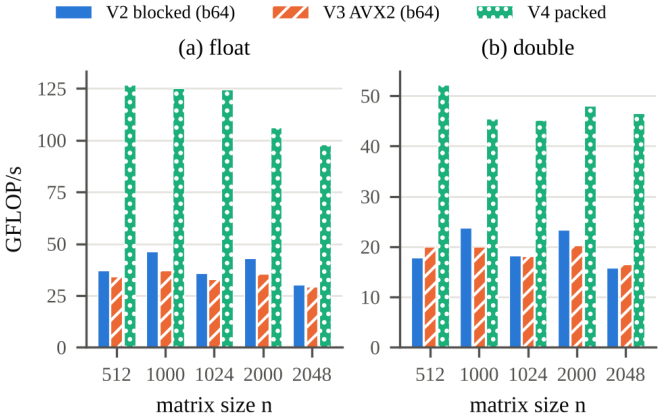


Fig. 5. EXP05. Blocked (V2), AVX2 (V3) and packed microkernel (V4) measured in one run, native build (median of 5). Run 20261010-172006.

TABLE VI
EXP05: PACKED MICROKERNEL (V4) THROUGHPUT

n	Float	% peak	× V2	Double	% peak	× V2
512	126.8	43.1	3.42	52.2	35.5	2.94
1000	125.0	42.5	2.70	45.4	30.9	1.91
1024	124.5	42.3	3.49	45.2	30.7	2.47
2000	106.3	36.1	2.48	48.1	32.6	2.06
2048	98.1	33.3	3.23	46.5	31.6	2.94

Throughput in GFLOP/s. Peak: 294.4 (float) and 147.2 (double) GFLOP/s at 4.6 GHz (Section III-E). "× V2": ratio to blocked_b64 in the same run.

The clearest effect of packing is on power-of-two sizes (Fig. 6). In the same run the blocked kernel loses 23% at n = 1024 and 29–32% at n = 2048 relative to the neighbouring non-power-of-two size, while the packed kernel loses at most 7.7%, and essentially nothing at n = 1024. This part of the prediction held: packed panels are contiguous buffers, so the 4 KiB row stride of the original matrices never reaches the inner loop.

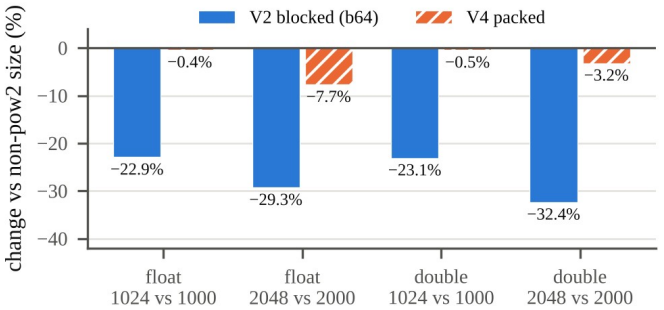


Fig. 6. Power-of-two penalty: change in throughput at n = 1024 and 2048 relative to n = 1000 and 2000, blocked (V2) versus packed (V4) kernel, same run (EXP05).

Two patterns were not predicted. First, the gain at n = 512 was not smaller than at larger sizes; float throughput is in fact highest there. Second, float throughput is flat near 125 GFLOP/s up to n = 1024 and falls to 106 and 98 GFLOP/s at n = 2000 and 2048, while double is already flat at 45–48 GFLOP/s from n = 1000. Both drops coincide with the packed B block (KC × N × element size) outgrowing the 1.25 MiB L2 cache:

about 1 MiB for float at n = 1024 versus 2 MiB at n = 2000, and 1 MiB for double at n = 512 versus 2 MiB at n = 1000. This suggests that a smaller NC, keeping packed B in L2, would recover the large-n loss; the hypothesis has not yet been tested.

F. Summary of the Ladder

Table VII collects the best measured throughput of each stage at n = 1024. These values come from different runs and, as EXP05 showed, absolute numbers can drift by about 1.4× between days, so the table indicates scale rather than exact ratios. Even so, the packed kernel is more than two orders of magnitude faster than the naive loop on the same data. Table VIII summarizes each prediction against its outcome.

TABLE VII
BEST THROUGHPUT PER STAGE AT N = 1024, NATIVE BUILD (GFLOP/S)

Stage	Kernel	Float	Double	Exp.
V0	ijk	0.53	0.55	02
V1	ikj	22.71	10.55	02
V2	best tile	28.46	14.12	03
V3	avx2_b64	23.36	12.86	04
V4	packed	**124.52**	**45.24**	05

Each column entry is the median from the run of the listed experiment; cross-run comparisons are approximate.

TABLE VIII
PREDICTIONS VERSUS OUTCOMES

Exp.	Prediction (summary)	Outcome
01	4 KiB-stride sizes far slower than neighbours	Held (5.0×, 5.4×)
02	ikj ≈ kij fastest, jki/kji slowest; native helps ikj, not ijk	Mostly held; kij < ikj at large n
03	Interior tile optimum; blocking wins at large n; power-of-two still slow	Held (BS 64–96; up to 4.0×)
04	V3 1.5–2.5× over portable; ties native within 10%	Partial; native code up to 19% faster
05	V4 3–5× over native V2; power-of-two penalty < 10%; smaller gain at n = 512	Partial; 1.9–3.5×; penalty ≤ 7.7%; n = 512 not smaller

V. DISCUSSION

A. What Mattered

Across the ladder, the steps that changed data movement mattered far more than those that changed instructions. Loop order, blocking and packing each altered which data are reused and how contiguously they are read, and each produced large gains or removed a cliff. Writing intrinsics without changing the data flow (V3) produced no gain over a good compiler, while the same intrinsics combined with register blocking and packing (V4) produced the largest gain. For student and practitioner code, the implication is to restructure loops and layouts first and to hand-vectorize only inside a register-blocked microkernel.

B. Threats to Validity

Platform. All measurements were made under WSL2 on a hybrid CPU. Hardware performance counters, thread pinning and frequency readings were unavailable, so P-core versus E-core placement, clock speed and TLB behaviour are unknown; the cache explanations above are therefore models consistent with the data, not direct measurements. *Drift.* Absolute throughput varied by about 1.4× between days for identical code; conclusions are drawn from within-run comparisons, and the cross-run summary in Table VII is approximate. *Generality.* Only square matrices, one compiler (GCC 15.2) and one machine were studied. *Implementation.* V4 allocates its packing buffers on every call, inside the timed region; this slightly understates its throughput. *Peak.* The peak estimate assumes the maximum turbo clock, which may not be sustained.

VI. CONCLUSION AND FUTURE WORK

A reproducible, step-by-step study of single-threaded GEMM on an Intel Core i5-13450HX showed that loop order alone spans 85×, cache blocking adds up to 4×, explicit AVX2 intrinsics add nothing over the compiler unless the data layout also changes, and a packed 6×16 register-blocked microkernel reaches about 43% of the estimated single-core peak while removing the power-of-two penalty. Recording predictions before measuring made the misses—kij versus ikj, the compiler beating intrinsics, and the day-to-day drift—as informative as the successes.

Future work will (i) parallelize V4 with OpenMP following [11] and study scaling across P- and E-cores; (ii) compare against an optimized BLAS library (OpenBLAS) at one and all threads; (iii) sweep NC to test the L2 hypothesis of EXP05; (iv) re-run all kernels in a single run for a drift-free final ladder; and (v) repeat the study on native Linux with hardware counters and core pinning to confirm the cache explanations directly.

REFERENCES

[1] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," *ACM Trans. Math. Softw.*, vol. 16, no. 1, pp. 1–17, Mar. 1990.

[2] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, Art. 12, May 2008.

[3] F. G. Van Zee and R. A. van de Geijn, "BLIS: A framework for rapidly instantiating BLAS functionality," *ACM Trans. Math. Softw.*, vol. 41, no. 3, Art. 14, Jun. 2015.

[4] T. M. Low, F. D. Igual, T. M. Smith, and E. S. Quintana-Ortí, "Analytical modeling is enough for high-performance BLIS," *ACM Trans. Math. Softw.*, vol. 43, no. 2, Art. 12, Aug. 2016.

[5] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*, Santa Clara, CA, USA, 1991, pp. 63–74.

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[7] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[8] R. Freivalds, "Probabilistic machines can use less running time," in *Proc. IFIP Congress 77*, Toronto, ON, Canada, 1977, pp. 839–842.

[9] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[10] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC15)*, Austin, TX, USA, 2015, Art. 73.

[11] T. M. Smith, R. A. van de Geijn, M. Smelyanskiy, J. R. Hammond, and F. G. Van Zee, "Anatomy of high-performance many-threaded matrix multiplication," in *Proc. IEEE 28th Int. Parallel and Distributed Processing Symp. (IPDPS)*, Phoenix, AZ, USA, 2014, pp. 1049–1059.

[12] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Santa Clara, CA, USA: Intel Corp.

[13] Notebookcheck, "Intel Core i5-13450HX processor – benchmarks and specs." [Online]. Available: https://www.notebookcheck.net/Intel-Core-

i5-13450HX-Processor-Benchmarks-and-Specs.677231.0.html (accessed Oct. 21, 2026).

[14] Pratham, “gemmx-cpp: A step-by-step study of single-threaded GEMM optimization,” GitHub repository, 2026. [Online]. Available: https://github.com/Pratham-2105/gemmx-cpp